

CHAPTER 07

필터링

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

7장 필터링

7.1 영상의 필터링

7.2 블러링: 영상 부드럽게 하기

7.3 샤프닝: 영상 날카롭게 하기

7.4 잡음 제거 필터링

7.2 블러링: 영상 부드럽게 하기

블러링: 영상 부드럽게 하기

- 블러링(blurring)은 마치 초점이 맞지 않은 사진처럼 영상을 부드럽게 만드는 필터링 기법이며 스무딩(smoothing)이라고도 함
- 영상에서 인접한 픽셀 간의 픽셀 값 변화가 크지 않은 경우 부드러운 느낌을 받을 수 있음
- 거친 느낌의 입력 영상을 부드럽게 만든다거나 입력 영상에 존재하는 잡음의 영향을 제거하는 전처리 과정으로도 사용됨



입력 영상



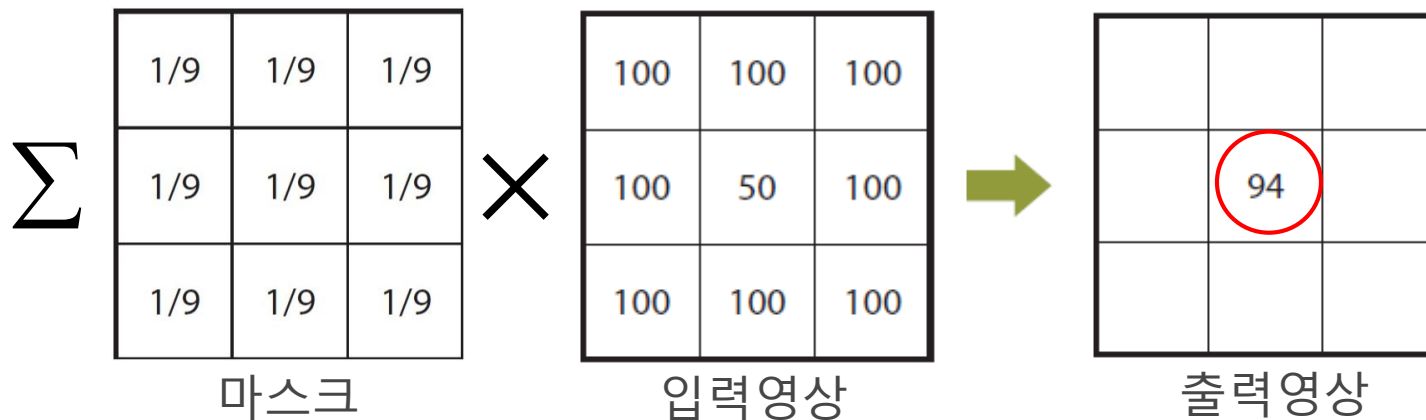
노이즈 있는 입력 영상



결과 영상

평균값 필터

- 입력 영상에서 특정 픽셀과 주변 픽셀들의 산술 평균을 결과 영상 픽셀 값으로 설정하는 필터임
- 결과 영상은 픽셀 값의 급격한 변화가 줄어들어 날카로운 에지가 무뎠지고 잡음의 영향이 크게 사라지는 효과가 있음
- 평균값 필터를 너무 과도하게 사용할 경우 사물의 경계가 흐릿해지고 사물의 구분이 어려워질 수 있음



$$(100 + 100 + 100 + 100 + 50 + 100 + 100 + 100 + 100) / 9 = 94.4$$

평균값 필터

- 마스크 행렬의 원소값 = $1/(\text{행렬의 전체 원소개수}) \rightarrow$ 마스크 원소의 합은 1임(평균 밝기를 동일하게 유지)
- 평균값 필터는 마스크의 크기가 커지면 커질수록 더욱 부드러운 느낌의 결과 영상을 생성하며 $\rightarrow$ 연산량이 크게 증가할 수 있음

$\frac{1}{9} \times$

1	1	1
1	1	1
1	1	1

$\frac{1}{25} \times$

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

그림 7-6 다양한 크기의 평균값 필터 마스크

평균값 필터

```
void blur(InputArray src, OutputArray dst, Size ksize,
          Point anchor = Point(-1,-1), int borderType = BORDER_DEFAULT);
```

- **src** 입력 영상. 다채널 영상은 각 채널별로 블러링을 수행합니다. 입력 영상의 깊이는 CV_8U, CV_16U, CV_16S, CV_32F, CV_64F 중 하나여야 합니다.
- **dst** 출력 영상. src와 같은 크기, 같은 채널 수를 갖습니다.
- **ksize** 블러링 커널 크기
- **anchor** 고정점 좌표. Point(-1, -1)을 지정하면 커널 중심을 고정점으로 사용합니다.
- **borderType** 가장자리 픽셀 확장 방식

- blur() 함수를 이용하여 평균값 필터링을 수행할 수 있고 filter2D 함수를 이용하여 구현 가능

평균값 필터

- blur() 함수는 src 영상에 ksize 크기의 평균값 필터 마스크를 사용하여 dst 출력 영상을 생성함
- anchor 인자와 borerType 인자는 기본값을 가지고 있으므로 함수 호출 시 생략할 수 있음
- blur() 함수에서 사용하는 커널(마스크)은 다음과 같은 형태를 가짐

$$\text{kernel} = \frac{1}{\text{ksize.width} \times \text{ksize.height}} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

- 마스크 행렬의 모든 원소의 합이 1이 되도록 설계함, 필터링 결과 영상의 평균 밝기가 입력 영상의 평균 밝기와 같게 유지됨

코드 7-2 : 평균값 필터

- 코드 7-2의 `blurring_mean()` 함수는 3×3 , 5×5 , 7×7 크기의 평균값 필터를 이용하여 `rose.bmp` 장미 영상을 부드럽게 변환하고 그 결과를 화면에 출력함

코드7-2 : 평균값 필터

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("rose.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}
    imshow("src", src);

    Mat dst;
    for (int ksize = 3; ksize <= 7; ksize += 2) {
        blur(src, dst, Size(ksize, ksize));

        String desc = format("Mean: %dx%d", ksize, ksize);
        putText(dst, desc, Point(10, 30), FONT_HERSHEY_SIMPLEX, 1.0,
                    Scalar(255), 1, LINE_AA);
        imshow("dst", dst);
        waitKey();
    }
    return 0;
}
```

코드7-2 : 평균값 필터



그림 7-7 평균값 필터를 이용한 블러링 실행 결과

가우시안 필터

- 가우시안 분포는 평균을 중심으로 좌우 대칭의 종 모양(bell shape)을 갖는 확률 분포를 말하며 정규 분포(normal distribution)라고도 함
- 자연계에서 발생하는 대부분의 사건은 가우시안 분포를 따르는 것으로 알려져 있음, 키, 몸무게, 시험점수 등의 분포
- 가우시안 분포는 평균과 표준 편차에 따라 분포 모양이 결정됨
- 영상의 가우시안 필터에서는 주로 평균이 0인 가우시안 분포 함수를 사용함
- 평균이 0이고 표준 편차가 σ 인 1차원 가우시안 분포를 함수식으로 나타내면 다음과 같음

$$G_{\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

가우시안 필터

- 가우시안 분포 함수 값은 특정 x 가 발생할 수 있는 확률의 개념을 가지며, 그래프 아래 면적을 모두 더하면 1이 됨
- 3개의 그래프가 모두 평균이 0이므로 $x = 0$ 에서 최대값을 가지며, x 가 0에서 멀어질수록 함수 값이 감소함
- 표준 편차 σ 가 작으면 그래프가 뾰족한 형태가 되고 표준 편차 σ 가 크면 그래프가 넓게 퍼지면서 완만한 형태

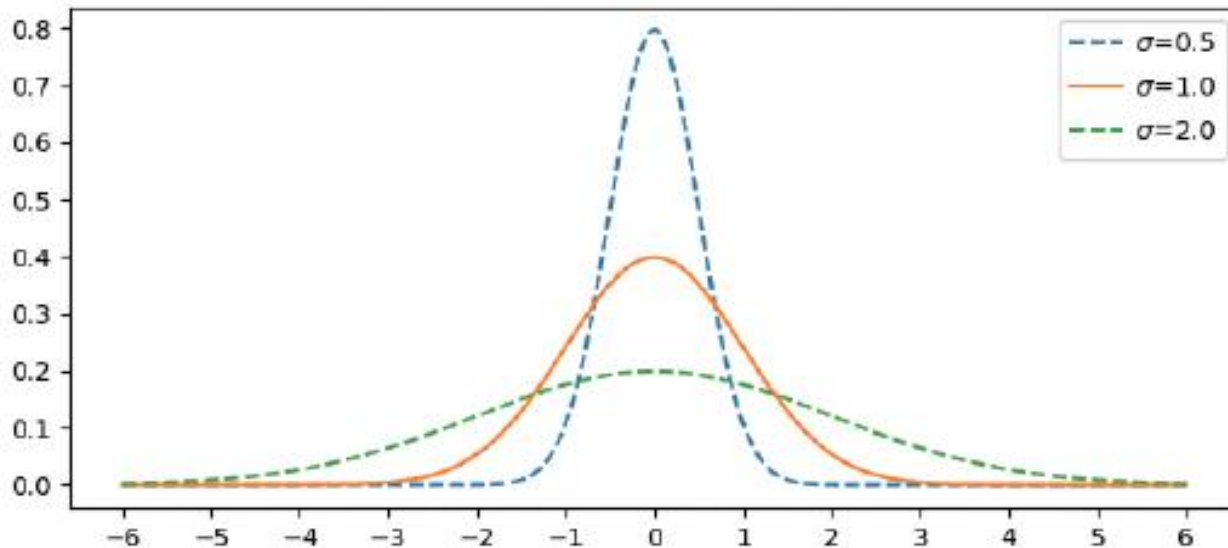


그림 7-8 평균이 0인 1차원 가우시안 분포 함수 그래프

가우시안 필터

- 가우시안 분포를 따르는 2차원 필터 마스크 행렬을 생성하려면 2차원 가우시안 분포 함수가 필요
- 2차원 가우시안 분포 함수는 x 와 y 두 개의 변수를 사용하고, 분포의 모양을 결정하는 평균과 표준 편차도 x 축과 y 축 방향에 따라 따로 설정함
- 평균이 $(0, 0)$ 이고 x 축과 y 축 방향의 표준 편차가 각각 σ_x , σ_y 인 2차원 가우시안 분포 함수는 다음과 같이 정의됨

$$G_{\sigma_x \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} e^{-\left(\frac{x^2}{2\sigma_x^2} + \frac{y^2}{2\sigma_y^2}\right)}$$

가우시안 필터

- 2차원 가우시안 분포 함수 그래프는 1차원 가우시안 분포 함수 그래프의 차원을 확장한 형태임
- 평균이 (0, 0)이므로 그림 7-9 그래프는 (0, 0)에서 최댓값을 갖고, 평균에서 멀어질수록 함수가 감소함
- 2차원 가우시안 분포 함수의 경우, 함수 그래프 아래의 부피를 구하면 1이 됨

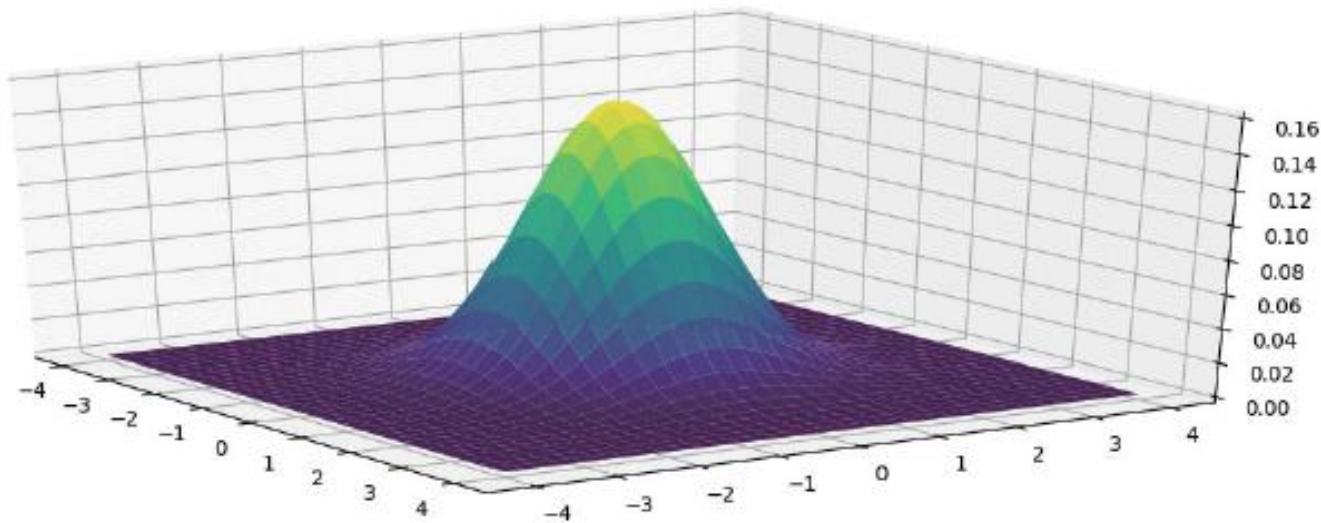


그림 7-9 평균이 (0, 0)인 2차원 가우시안 함수 그래프($\sigma_x = \sigma_y = 1.0$)

가우시안 필터

- 가우시안 필터는 2차원 가우시안 분포 함수로부터 구한 마스크 행렬을 사용함 -> 평균값 필터보다 자연스러운 블러링 효과 얻음
- 가우시안 잡음을 제거하는 데 효과적
- 가우시안 분포 함수는 연속 함수이지만 이산형의 마스크를 만들기 위해서 x 와 y 값이 정수인 위치에서만 가우시안 분포 함수 값을 추출하여 마스크를 생성함
- 평균이 0이고 표준 편차가 σ 인 가우시안 분포는 x 가 -4σ 부터 4σ 사이인 구간에서 그 값의 대부분이 존재하기 때문에 가우시안 필터 마스크의 크기는 보통 $(8\sigma+1)$ 로 결정함

가우시안 필터

$$G = \begin{pmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0001 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0002 & 0.0011 & 0.0018 & 0.0011 & 0.0002 & 0.0000 & 0.0000 \\ 0.0000 & 0.0002 & 0.0029 & 0.0131 & 0.0215 & 0.0131 & 0.0029 & 0.0002 & 0.0000 \\ 0.0000 & 0.0011 & 0.0131 & 0.0586 & 0.0965 & 0.0586 & 0.0131 & 0.0011 & 0.0000 \\ 0.0001 & 0.0018 & 0.0215 & 0.0965 & 0.1592 & 0.0965 & 0.0215 & 0.0018 & 0.0001 \\ 0.0000 & 0.0011 & 0.0131 & 0.0586 & 0.0965 & 0.0586 & 0.0131 & 0.0011 & 0.0000 \\ 0.0000 & 0.0002 & 0.0029 & 0.0131 & 0.0215 & 0.0131 & 0.0029 & 0.0002 & 0.0000 \\ 0.0000 & 0.0000 & 0.0002 & 0.0011 & 0.0018 & 0.0011 & 0.0002 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0001 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{pmatrix}$$

그림 7-10 $\sigma_x = \sigma_y = 1.0$ 인 경우의 가우시안 필터 마스크(9x9)

가우시안 필터

- 필터링 대상 픽셀 근처에는 가중치를 크게 주고 필터링 대상 픽셀과 멀리 떨어져 있는 주변부에는 가중치를 조금만 주어서 가중 평균(weighted average)을 구하는 것과 같음
- 가우시안 필터 마스크가 가중 평균을 구하기 위한 가중치 행렬 역할을 하는 것임
- 마스크 연산에 의한 영상 필터링은 마스크 크기가 커짐에 따라 연산량도 함께 증가함 그림 7-10의 경우, 마스크 행렬 크기가 9×9 이기 때문에 연산 시 81번의 곱셈 연산이 필요함
- 큰 표준 편차 값을 사용하면 마스크 크기도 함께 커지므로 연산 속도 측면에서 부담이 될 수 있음

가우시안 필터

```
void GaussianBlur(InputArray src, OutputArray dst, Size ksize,
                  double sigmaX, double sigmaY = 0,
                  int borderType = BORDER_DEFAULT);
```

- **src** 입력 영상. 다채널 영상은 각 채널별로 블러링을 수행합니다.
- **dst** 출력 영상. src와 같은 크기, 같은 타입을 갖습니다.
- **ksize** 가우시안 커널 크기. ksize.width와 ksize.height는 0보다 큰 홀수이어야 합니다. ksize에 Size()를 지정하면 표준 편차로부터 커널 크기를 자동으로 결정합니다.
- **sigmaX** x 방향으로의 가우시안 커널 표준 편차
- **sigmaY** y 방향으로의 가우시안 커널 표준 편차. 만약 sigmaY = 0이면 sigmaX와 같은 값을 사용합니다. 만약 sigmaX와 sigmaY가 모두 0이면 ksize의 width와 height 값으로부터 표준 편차를 계산하여 사용합니다.
- **borderType** 가장자리 픽셀 확장 방식

- 가우시안 필터링을 수행하려면 GaussianBlur() 함수를 사용하고 filter2D함수를 이용하여 구현 가능

코드 7-3 : 가우시안 필터

- 코드 7-3의 `blurring_gaussian()` 함수는 가우시안 표준 편차를 1부터 5까지 정수 단위로 증가시키면서 `rose.bmp` 장미 영상에 대해 가우시안 필터링을 수행함

코드7-3 : 가우시안 필터

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("rose.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}
    imshow("src", src);

    Mat dst;
    for (int sigma = 1; sigma <= 5; sigma++) {
        GaussianBlur(src, dst, Size( ), (double)sigma);

        String desc = format("Gaussian: sigma = %d", sigma);
        putText(dst, desc, Point(10, 30), FONT_HERSHEY_SIMPLEX, 1.0,
                    Scalar(255), 1, LINE_AA);
        imshow("dst", dst);
        waitKey();
    }
    return 0;
}
```

코드7-3 : 가우시안 필터

- 표준 편차 값이 커질수록 결과 영상이 더욱 부드럽게 변경되는 것을 확인할 수 있음

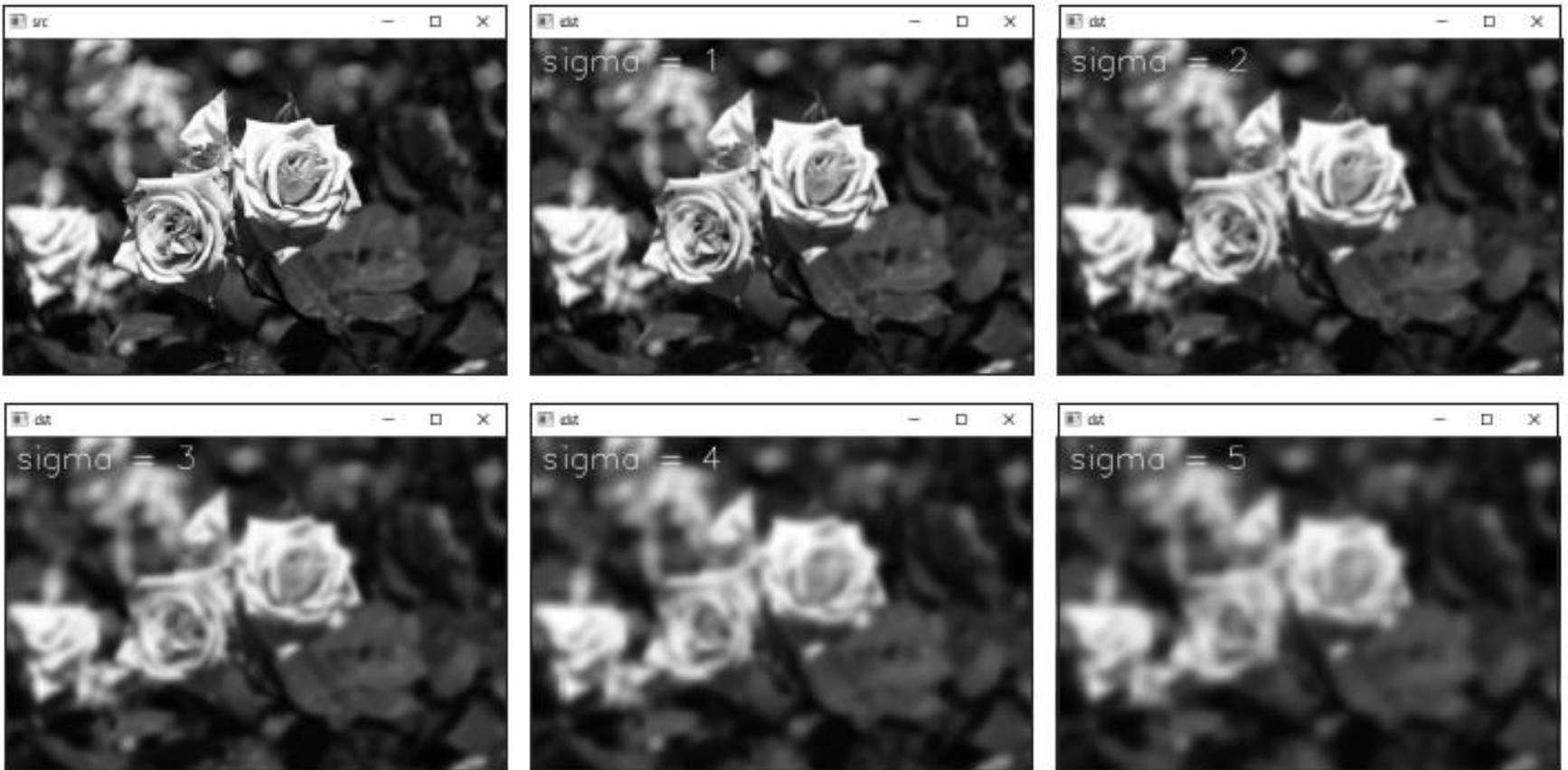


그림 7-11 가우시안 필터링 예제 실행 화면

실습과제 1

- 코드 7-2에서 filter2D 함수를 이용하여 평균값 필터(blur)를 구현하시오.

실습과제2

- 평균값 필터의 마스크 사이즈가 3×3 , 10×10 , 30×30 일때 함수의 실행시간을 비교하시오.
- 실행시간 차이가 없으면 마스크 사이즈를 더 증가시켜보라.
- 실행시간 측정방법1: C++라이브러리 함수 이용

```
#include <ctime>           //<time.h>
clock_t start = clock();    //시작시간 저장(단위:msec)
// 실행시간을 측정할 코드
clock_t end = clock();      // 종료시간 저장(단위:msec)
cout << "실행시간:" << (end - start) << "msec" << endl;
```

- 실행시간 측정방법2 : 교재 184페이지 TickMeter 클래스 이용

실습과제3

- filter2D 함수를 이용하여 가우시안 필터를 구현 하시오. 마스크 행렬은 다음을 사용하라.
- GaussianBlur 함수에서 ksize 를 5x5 로 설정했을 때와 결과를 비교해보시오.

$$\frac{1}{273}$$

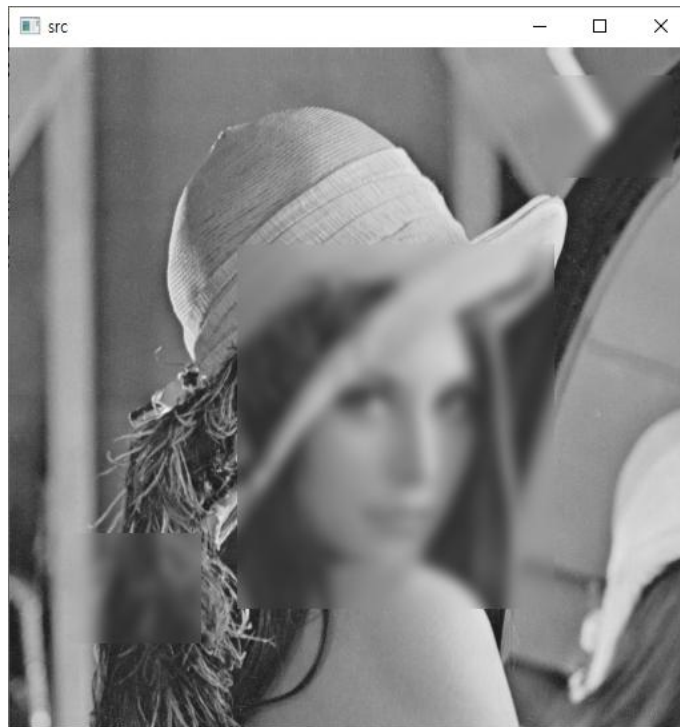
1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

실습과제4

- 코드 7-3에서 표준편차가 1~5일때 각각의 GaussianBlur함수의 실행시간을 구하여 출력하시오.

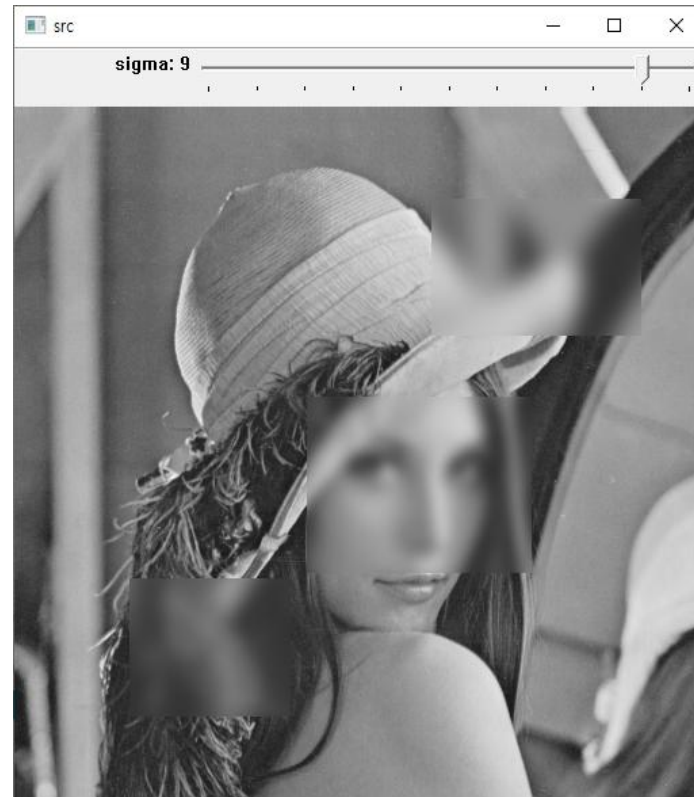
실습과제5

- lenna영상을 그레이로 변환하여 출력하고 마우스이벤트를 이용하여 마우스로 드래깅한 영역만 가우시안 블러링을 수행하는 프로그램을 작성하시오. 가우시안 블러함수의 표준편차는 5로 설정하라.



실습과제6

- 실습5에서 가우시안 블러함수의 표준편차를 트랙바를 이용하여 선택할수 있도록 하시오. 트랙바의 범위는 0~9으로 설정하라. 표준편차의 초기값은 반드시 1로 설정하라.



과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.